Implementation of FTP Server using C on Linux

Project Code : 80788
Submitted By : Rajat Srivastava – E2 – 80788
Under the Guidance of : Sohail Inamdar
Institute : Sunbeam Infotech Private Limited
Date : 04 September 2025

2. Abstract

The File Transfer Protocol(FTP) is a standard network protocol used for the transfer of computer files betweeen a client and server on a computer network. This project aimed to design and implement a custom, minimal FTP server from the ground up using the C programming language on a Ubuntu. The motivation was to gain a deep, practical, understanding of network programming, the TCP/IP socket API, and the intricacies of a classic application-layer protocol.
The key modules implemented include a main server daemon that listens for incoming connections, a robust user authentication system, and handlers for core FTP commands like **USER, PASS, LIST, RETR**(download), **STOR**(upload) and **QUIT.** The server efficiently handles multiple clients simultaneously by forking a new process for each connection. The main outcomes were a functiona, command-line FTP server capable of secure user login, directory listing, and reliable file uploads and downloads, demonstating the fundamental principle of client-server architecture and TCP-based data transfer.

3. Introduction

FTP is one of the oldest and most reliable protocols for transferring files over a network. It operated on a client-server model and uses a seperate TCP connection for control(command port 21) and data transfer(dynamically assigned port). Despite the rise of HTTP and more secure alternatives like SFTP, FTP remains relevant in many scenarios, such as website maintenance, internal file sharing in isolated networksm and automated data exchange systems.
The primary objective of this project was to build a minimal yet functional FTP sever that supports core RFC 959 commands. The key goal included implementing user authentication to secure access, supporting basic file operations like listing directory contents and transferring files, and designing a concurrent server architecture to handle multiple clients at once without blocking. This project serves as a practical exploration of network socket programming in C.

4. Literature Review / Background

FTP was defined in 1971 and standardized in 1985 by RFC 959. It was designed for simplicity and efficiency in an era before security was a primary concern.
Unlike HTTPm which is stateless and uses single connection, FTP is a stateful and uses dual channels, making it more for large file transfer but complex to implement and firewall.
Modern production system often use sophisticated server like **vsftpd**(very secure FTP daemon) or **proftpd,** which offers high performance, extensive security features, and configurability. This project, however, involves building a custom server. The purpose is not to compete with these established tools but to demystify their core functionality and provide a hands-on learning experience in protocol implementation and concurent network programming.

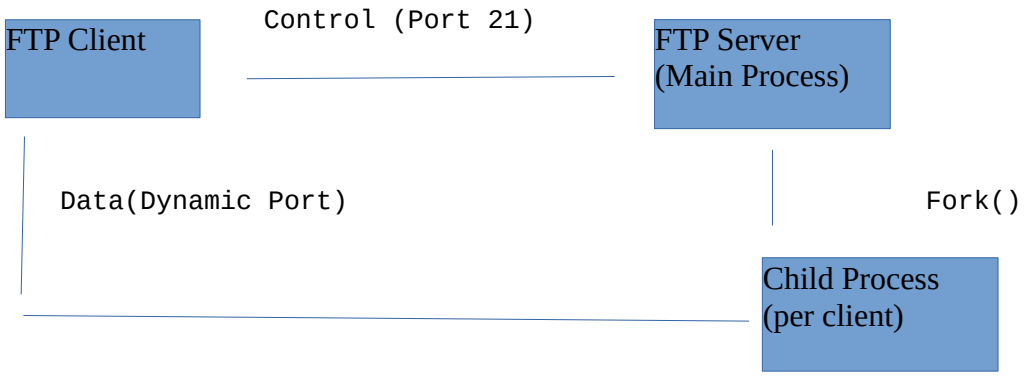
5. System Requirements

Hardware Requirements:
1. Standard PC or Laptop.
2. Minimum 4GB RAM.

Software Requirements:
1. Operating System : Ubuntu 22.04 LTS.
2. Compiler: GCC (GNU Compiler Collection).
3. Developmenet Libraries : Standard C library with Socket API(sys/socket.h, netinet/in.h).

4. Analysis Tool : Wireshark for capturing and analyzing network packets.

6. System Desing



Flowchart of Request-Response:

1. Server Setup : Main process creates socket, binds to port 21, and calls listen().
2. Client Connection : Client connects to server port 21. Main process accept() the connection.
3. Process Creation : Main process calls fork(). The parent closes the new socket and goes back to accept(). The child process becomes the handler for this client.
4. Authentication : Child process sends welcome banner. Client send **USER** and **PASS.** Child validates against credentials and grant access.
5. Command Loop :
    A. Child wait for a command on the control socket.
    B. Upon receiving a command(e.g., LIST), it parses it.
    C. For data commands, it establishes data connection(open a data socket, calculates PASV port, send port info to client, waits for clients data connection).
    D. Executes the command(e.g., runs ls -l and sends output over data socket).
    E. Sends a completion status (e.g., “226 Transfer complete”) on the control socket.
    F. Returns to wait for the next commands.

File Transfer Modes:

> Connection Mode : Passive (PASV) mode was implemented, where the client initiates both control and data connections to the server. This is more firewall-friendly.
> Data Type : Binary(TYPE I) mode was implemented to handle all files types without corruption.

7. Implementation

The server was developed using C and the socket API on Linux.

1. Socket Creation & Binding : The main server creates a TCP socket, binds it to port 21, and listens for connections.

.....C

```
int socket_fd = socket(AF_INET, SOCK_STREAM, 0);
int optval = 1;
setsockopt(socket_fd, SOL_SOCKET, SO_REUSEADDR, (const void *)&optval, sizeof(int));
struct sockaddr_in serv_addr;
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = INADDR_ANY;
serv_addr.sin_port = htons(21);

bind(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
listen(sockfd, 5);
```

2. User Authentication : A simple username / password mechanism is hardcoded for

validation (e.g., user : “rajat”, password : “80788”).

```C
      if(strcmp(received_username, “rajat”) == 0 && strcmp(received_password, “80788”) == 0){
            send(client_sock, “230 Login successful.\r\n”, ....);
      }
      else{
            send(client_sock, “530 Login incorrect.\r\n”, ....);
}
```

3. Handling FTP Commands : The child process uses a loop to read commands from the control socket and calls corresponding functions.

```C
      char buffer[256];
      read(control_sock, buffer, 255);
      if(strncmp(buffer, “RETR”, 4) == 0){
            handle_retr(control_sock, buffer+5);
      } elseif(strncmp(buffer, “LIST”, 4) == 0){
            handle_list(control_sock);
      }
```

4. Handling RETR Command :

```C
      void handle_retr(int ctrl_sock, int data_sock, const char *filename){
      FILE *file = fopen(filename, “rb”); //open in binary mode
      if(!file){
            send_response(ctrl_sock, “550 file not found.\r\n”).
            return;
      }
      send_response(ctrl_sock, “150 Opening binary mode data connection for file transfer.\r\n”);
      char_buffer[BUFFER_SIZE];
      size_t bytes_read;
      while((bytes_read = fread(buffer, 1, BUFFER_SIZE, file)) > 0){
            send(data_sock, buffer, bytes_read, 0);
      }
      fclose(file);
      close(data_sock);
      send_response(ctrl_sock, “226 file transfer successful\r\n”).
      }
```

8. Testing & Result

1. Authentication :
2. File Download(RETR) :
3. File Upload(STOR) :
4. Directory Listing(LIST) :
5. Concurrency :

9. Chanllenges & Solutions

-> Challenge : “Address already in use” error. This occurred when restarting the server quickly because the socket was in a TIME_WAIT state.

Solution : Used the setsocket() API with SO_REUSEADDR to allow the socket to allow the socket to be reused.

10. Applications

Lightweight Internal Tool : Can be used for simple, secure file sharing within a trusted local network.

Embedded Systems : The minimal and customizable nature of the server makes it suitable for resource-constrained environments where a full-blown vsftpd might be unnecessary, such as in IoT devices for firmware updates or data retrieval.

12. Conclusion

This project provided invaluable hands-on experience in building a fundamental internet service from scratch. It deepened the understanding of the TCP/IP protocol suite, specifically the socket programming interface, process management, and application layer FTP protocol. The successful implementation of a multi-client FTP server demonstrates a strong grasp of key networking concepts. The skills honed during this project – C programming, socket API usage, debugging complex network interactions, and protocol analysis with Wireshark – are fundamental and directly transferable to developing any network based application. This project solidifies one’s understanding of how foundational internet protocol operates at the application level.

13.References

1. J. Postel and J. Reynolds, “FILE TRANSFER PROTOCOL”, RFC 959, October 1985.
2. Stevens, W. Richard, et al. "Unix Network Programming, Volume 1: The Sockets Networking API." Addison-Wesley Professional, 2003.

3. Linux man pages: man 2 socket, man 2 bind, man 2 listen, man 2 accept, man 2 fork.

4. GNU C Library Manual: https://www.gnu.org/software/libc/manual/

5. Beej's Guide to Network Programming: https://beej.us/guide/bgnet/

6. vsftpd documentation: https://security.appspot.com/vsftpd.html